LAB 1 — File descriptors y durabilidad

> **1 sesión (2 h).** Entorno: **VM Linux** (no Docker). Presupuesto: **N1 ≈ 90 min** (núcleo) · **N2 ≈ 20 min** (si alcanza el tiempo) · **N3 = tarea** (requiere reinicio). Dos escenarios: fuga de FDs (502) y pérdida de datos (fsync).

LAB 1: File descriptors y durabilidad (`Nginx` 502 · `MySQL` fsync)

Duración: 1 sesión (2 h) · **Tema de Syllabus:** 4.1, 4.2 · **Entorno:** VM Linux (sudo)
Herramientas: `/proc`, `ulimit`/`sysctl`, `strace`, `dd`, `python3`
Niveles: N1 *Resolver* (guiado, en sesión) · N2 *Diagnosticar* (sin receta) · N3 *Runbook* + *alerta* (fuera de sesión)

> ### DOS ESCENARIOS A RESOLVER
>
> **Escenario 1 — Fuga de FDs (Nginx 502).**
> **Ticket #1043 (02:14):** El Nginx responde `502` y aparece `24: Too many open files` en `error.log`. La CPU está al 5 % y la red está normal.
>
> **Escenario 2 — Pérdida de datos (MySQL).**
> **Ticket #1044 (01:50):** Tras un reinicio duro, MySQL perdió transacciones que la app daba por confirmadas.
>
> **Objetivo:** resolver cada escenario por separado. Para cada caso: hipótesis → comando → resultado → decisión → acción.
>
> **Evidencia:** podés incluir **capturas de pantalla** que muestren el comando y su salida, y una explicación de lo que hiciste. Eso basta para demostrar que lo ejecutaste.

> **Entorno — por qué VM y no Docker.** Los experimentos tocan estado global del kernel (`fs.file-max`) o reinician la máquina (caída dura): eso no tiene *namespace* y en un contenedor afectaría al host. Si Nginx/MySQL no están instalados, usá un proceso de prueba equivalente (se indica en cada paso); la guía de mapeo servicio→concepto está en `../guias/GUIA_ENCAMINO_SERVICIOS_SO.md`.

Kit mínimo (y por qué solo esto)

Herramienta	Para qué	Pregunta que responde
`/proc/<pid>/fd`, `/proc/<pid>/limits`	descriptores y techo del proceso	¿cuántos FDs usa y con qué límite?
`ulimit`, `/proc/sys/fs/file-max`	límite por proceso vs global	¿se agotó el proceso (`EMFILE`) o el kernel (`ENFILE`)?
`strace -c`	patrón de syscalls	¿abre sin cerrar?
`/proc/meminfo` + `dd`	page cache y durabilidad	¿el dato llegó al disco antes del `return`?

> El resto (`fincore`, `sysrq`, `dumpe2fs`, `drop_caches`…) **no se ejecuta por defecto**: es *ampliación* y entra solo si el síntoma lo pide.

N1 — Reproducir el `502` y arreglarlo

P1 — ¿Cuántos FDs usa el proceso y cuál es su techo?

> Escribí tu hipótesis *antes* de medir: ¿el límite es del **proceso** o del **kernel**?

```
python3 -c 'import time; time.sleep(600)' &
P=$!
ls /proc/$P/fd | wc -l
```

```
grep 'open files' /proc/$P/limits
```

```
cat /proc/sys/fs/file-nr
```

```
cat /proc/sys/fs/file-max
```

Decisión: si los FDs se acercan a `Max open files` → problema **del proceso** (`EMFILE`, el error `24` de Nginx). Si se acercan a `file-max` → problema **global** (`ENFILE`). Son incidentes distintos.

P2 — ¿Por qué el proceso se queda sin descriptores?

```
( ulimit -Sn 1024; for i in $(seq 1 5000); do exec {fd}<>/dev/null || { echo "EMFILE en $i";
break; }; done )
```

```
ls /proc/$BASHPID/fd 2>/dev/null | wc -l
```

Conclusión (escribila): falla cerca de 1024 = tu *soft limit* ⇒ el `24` del ticket es **`EMFILE`**, no una caída de red.

P3 — ¿El código abre sin cerrar?

```
python3 -c 'import os,time
while True:
    try: os.open("/etc/hosts",0); time.sleep(0.02)
    except OSError: break' &
P=$!
```

```
strace -f -c -p $P 2>&1 | grep -E 'openat|close'
```

```
kill $P 2>/dev/null
```

Decisión: `openat` ≫ `close` ⇒ **fuga**. En C es un `open()` cuyo `close()` queda fuera del `finally`.

P4 — Aplicá la remediación y verificá.

```
ulimit -n 65535
```

```
for i in $(seq 1 20000); do exec {fd}<>/dev/null; done; echo "OK"
```

```
ls /proc/$$/fd | wc -l
```

Decisión: con el límite elevado ya no falla ⇒ la causa era el **límite del proceso**, no el kernel.

> **Runbook (boceto N1):**
>
> | Síntoma | Comando | Lectura | Acción |
> |---|---|---|---|
> | `502` + `24 Too many open files` | `grep 'open files' /proc/<pid>/limits` | FDs usados ≈ `Max open files` | subir `LimitNOFILE` (systemd) **y** cerrar FDs en el código |
> | `fs.file-nr` cerca del máximo | `cat /proc/sys/fs/file-nr` | agotamiento **global** (`ENFILE`) | subir `fs.file-max` (no es este ticket) |

N2 — ¿Quién se queda sin FDs?

> **Escenario.** Hay **3 procesos "app"** (1 sano, 2 con fuga) y **no** sabés cuál. **No hay receta ni `lsof`.** Elegí las herramientas y justificá.

Tu informe debe responder:

1. **Hipótesis previa** (quién y por qué) y **cómo** la verificaste (qué comando elegiste y por qué ese).
2. **Qué recurso** agota el culpable (¿sockets a `:3306`? ¿archivos?).
3. **Remediación en 3 capas:** código (`close()` en `finally`), proceso (`LimitNOFILE` en systemd), kernel (`fs.file-max`).

> Se evalúa el **método**, no la captura. *Pista de servicio:* en una API Node el leak típico es un pool de MySQL sin `close()` en la ruta de error; en Nginx, `worker_connections` y `LimitNOFILE`.

N3 (Runbook + alerta) — "Se perdieron datos": `write()` vs `fsync`

> **Ticket #1044.** Tras un reinicio duro, MySQL perdió transacciones que la app creía confirmadas. Diseñá el experimento que lo explica y entregá el runbook. **Requiere caída dura (reinicia la VM) ⇒ fuera de sesión.**

P1 — ¿Por qué `write()` retorna *antes* de que el dato esté en disco?

```
dd if=/dev/zero of=/tmp/a bs=1M count=200 status=none
grep -E '^(Dirty|Writeback):' /proc/meminfo    # el dato vive en RAM, no en el disco
```

P2 — Crear los archivos

```
R=/mnt/prueba
dd if=/dev/urandom of=$R/a.bin bs=1M count=64 conv=fsync status=none
```

```
sha256sum $R/a.bin | tee /tmp/hashes.txt
```

```
dd if=/dev/urandom of=$R/b.bin bs=1M count=64 status=none
sha256sum $R/b.bin >> /tmp/hashes.txt
```

P3 — Ver estado y provocar la caída

```
grep '^Dirty:' /proc/meminfo
```

```
echo b | sudo tee /proc/sysrq-trigger
```

P4 — Tras el reboot

```
sudo mount /dev/sdX /mnt/prueba && sha256sum -c /tmp/hashes.txt
```

```
dmesg | grep -iE 'ext4|recovery' | tail
```

P3 — El runbook: completá la fila y justificá con el mecanismo (`data=ordered` en ext4, *torn write*, dirty pages pendientes).

Síntoma	Comando	Lectura	Acción
datos perdidos tras un *crash*	`grep '^Dirty:' /proc/meminfo` antes de caer	`Dirty > 0` al momento de la caída	exigir `fsync` / `O_SYNC` en la ruta de *commit* (o replicar)

> **Variante segura (sin reinicio):** imagen de FS en loop, escribir con/sin `fsync`, **no** desmontar limpio y analizar con `dumpe2fs -h` + `debugfs -R 'ls <dir>'` qué llegó al *journal*.

> **Concepto:** `write()` escribe en el **page cache** (RAM); el SO hace *writeback* diferido. `fsync()` fuerza persistencia en disco y explica por qué MySQL confirma transacciones solo tras `fsync`.

Preguntas de anclaje

1. En P2, ¿por qué el error es `EMFILE` y no `ENFILE`? ¿Qué `sysctl` produciría `ENFILE`?
2. ¿Por qué dos rutas al mismo archivo comparten el *offset* (mismo `struct file`) pero **no** se cierran juntas?
3. En N3, ¿por qué el hash guardado **sí** sobrevive pero el archivo sin `fsync` puede no hacerlo?
4. Nombra la estructura del kernel detrás de cada síntoma: `502` → ; pérdida de datos → .

Entregables por nivel:

Nivel	Entregable	Formato
N1	Runbook del `502` + las 4 *decisiones* (P1–P4) con su número	tabla + 1 párrafo por decisión
N2	Método de identificación del culpable + remediación en 3 capas	informe breve
N3	Runbook de durabilidad + diseño del experimento + resultado	tabla + análisis

Rúbrica por niveles (máx. 10 pts):

Nivel	Criterios	Pts
N1 (6)	Reproduce y explica `EMFILE` vs `ENFILE` (2) · confirma la fuga con `strace` (1) · aplica y **verifica** la remediación (1) · runbook con comando+lectura+acción (2)	6
N2 (2)	Método propio (hipótesis + herramienta justificada) · identifica el recurso agotado y justifica las 3 capas (2)	2
N3 (2)	Experimento correcto + explica durabilidad (dirty/writeback/ `data=ordered`) + runbook con métrica y umbral (2)	2

Todo debe estar evidenciado con las capturas